

Advanced / Custom RAG Engineering

- Design production-grade RAG architectures.
- Implement hybrid retrieval and metadata filtering strategies.
- Optimize content usage and retrieval performance.
- Build enterprise-level knowledge retrieval systems.
- Pinecone
- Chroma
- Hybrid Search (BM25 + Vector)
- Re-ranking models
- Hybrid retrieval strategies
- Metadata filtering
- Query rewriting
- Content window optimization
- Structured/unstructured RAG
- Multi-source ingestion
- RAG over APIs
- Embedding optimization
- Retrieval performance tuning

Hybrid Search (BM25 + Vector Search)

What is Hybrid Search?

Hybrid Search is a retrieval technique that **combines Keyword Search (BM25) and Semantic Search (Vector Search)** to retrieve the most relevant documents. Instead of relying only on exact keyword matching or only on semantic similarity, Hybrid Search uses **both methods together** to improve search accuracy.

Definition

Hybrid Search is the process of combining sparse retrieval (BM25) and dense retrieval (vector embeddings) to return more accurate and relevant search results.

Why Do We Need Hybrid Search?

The single search technique is perfect.

BM25 Search (BM25)-best matching

BM25 searches for exact words **present** in the documents.

Example:

Documents

1. Pinecone is a vector database.

2. BM25 is a keyword ranking algorithm.

3. FAISS is a in-memory vector database.

Query:

What is BM25?

BM25 finds the exact word "BM25", so it retrieves:

BM25 is a keyword ranking algorithm.

This is correct.

Problem with BM25

Suppose the user searches

Semantic Search Database

Document:

Pinecone is a vector database.

Since the document doesn't contain the exact words **Semantic** or **Search**, BM25 may fail to retrieve it.

2. Vector Search

Vector Search converts text into **embeddings** (vectors) and searches based on **meaning**, not exact words.

Example:

Document:

Pinecone is a vector database.

Query:

Semantic Search Database

Even though the words are different, the meanings are similar.

Vector Search understands:

Semantic Search Database

Vector Database

So it retrieves the correct document.

Problem with Vector Search

Suppose the query is

ERROR

Document

Error Code ERROR occurs due to authentication failure.

Vector Search may not prioritize this document because **ERROR** is a specific keyword, not a semantic concept.

BM25 performs much better in such cases.

Solution: Hybrid Search

Hybrid Search combines the strengths of both approaches.

User Query

Hybrid Search

BM25

Vector Search

Merge Scores

Top Relevant Documents

Comparison

Feature

BM25

Vector Search

Hybrid Search

Exact keyword matching

Semantic understanding

Handles synonyms

Product IDs / Error Codes

Natural language queries

Overall accuracy

Medium

High

Very High

Who decides the BM25 and Vector scores?

The BM25 algorithm automatically considers the keyword frequency, inverse document frequency, and document length.

The vector database automatically computes the semantic similarity, producing a similarity metric such as cosine similarity between the query embedding and document embeddings.

The developer decides how to combine these scores (for example, by choosing the alpha weight, and in production this weight is typically optimized through experimentation or replaced with more advanced ranking techniques like LLM or learning-to-rank models).

What is Re-ranking?

Definition

Re-ranking is the process of re-evaluating the retrieved documents using a more powerful model to improve their order before sending them to the LLM.

Think of it as a second round of judging.

Example

Suppose the user asks:

How does Hybrid Search work?

After Hybrid Search, we get:

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Document

Imagine a Class of 5 Students

You ask everyone to write an essay.

Document 1

AI AI Machine Learning

Document 2

Machine Learning Python

Document 3

Cricket Football Tennis

Document 4

AI Machine Python

Document 5

Cooking Recipes Food

Now someone searches for

Step 1: Term Frequency (TF)

Ask students:

How many times does "AI" appear inside each document?

Document

AI Count (TF)

Document 1

Document 2

Document 3

Document 4

Document 5

Document 6

Document 7

Document 8

Document 9

Document 10

Document 11

Document 12

Document 13

Document 14

Document 15

Document 16

Document 17

Document 18

Document 19

Document 20

Document 21

Document 22

Document 23

Document 24

Document 25

Document 26

Document 27

Document 28

Document 29

Document 30

Document 31

Document 32

Document 33

Document 34

Document 35

Document 36

Document 37

Document 38

Document 39

Document 40

Document 41

Document 42

Document 43

Document 44

Document 45

Document 46

Document 47

Document 48

Document 49

Document 50

Document 51

Document 52

Document 53

Document 54

Document 55

Document 56

Document 57

Document 58

Document 59

Document 60

Document 61

Document 62

Document 63

Document 64

Document 65

Document 66

Document 67

Document 68

Document 69

Document 70

Document 71

Document 72

Document 73

Document 74

Document 75

Document 76

Document 77

Document 78

Document 79

Document 80

Document 81

Document 82

Document 83

Document 84

Document 85

Document 86

Document 87

Document 88

Document 89

Document 90

Document 91

Document 92

Document 93

Document 94

Document 95

Document 96

Document 97

Document 98

Document 99

Document 100

Document 101

Document 102

Document 103

Document 104

Document 105

Document 106

Document 107

Document 108

Document 109

Document 110

Document 111

Document 112

Document 113

Document 114

Document 115

Document 116

Document 117

Document 118

Document 119

Document 120

Document 121

Document 122

Document 123

Document 124

Document 125

Document 126

Document 127

Document 128

Document 129

Document 130

Document 131

Document 132

Document 133

Document 134

Document 135

Document 136

Document 137

Document 138

Document 139

Document 140

Document 141

Document 142

Document 143

Document 144

Document 145

Document 146

Document 147

Document 148

Document 149

Document 150

Document 151

Document 152

Document 153

Document 154

Document 155

Document 156

Document 157

Document 158

Document 159

Document 160

Document 161

Document 162

Document 163

Document 164

Document 165

Document 166

Document 167

Document 168

Document 169

Document 170

Document 171

Document 172

Document 173

Document 174

Document 175

Document 176

Document 177

Document 178

Document 179

Document 180

Document 181

Document 182

Document 183

Document 184

Document 185

Document 186

Document 187

Document 188

Document 189

Document 190

Document 191

Document 192

Document 193

Why is Re-ranking needed?

Vector Search only compares embeddings. It doesn't deeply understand the question. Example:
Query
How does Hybrid Search work?
Document A
Hybrid Search combines BM25 and Vector Search.
Document B
Pinecone pricing for Hybrid Search.
Both contain
Hybrid Search
Vector Search may rank them similarly.
A re-ranker reads the **entire question and document together**, realizing that Document A actually explains **how it works**, while Document B is about pricing.

Retrieval Pipeline



What Models Are Used for Re-ranking?

The most common are **Cross-Encoder** models. Unlike embedding models, which encode the query and document separately, a Cross-Encoder processes them together.
Query
+
Document
↓
Transformer
↓
Relevance Score

Popular Re-ranking Models

1. BAAI/bge-reranker-large

Very popular.
From sentence_transformers import CrossEncoder
model = CrossEncoder("BAAI/bge-reranker-large")
Example
score = model.predict(["What is Hybrid Search?", "Hybrid Search combines BM25 and Vector Search."])
Output
0.98

2. BAAI/bge-reranker-base

Smaller and faster.
Good for local applications.

3. Cohere Rerank API

Cloud API.
Query
↓
Cohere
↓
Top documents reordered
Very common in enterprise RAG.

Difference Between Embedding Model and Re-ranking Model

Embedding Model	Re-ranking Model
Encodes documents once and stores vectors.	Reads the query and document together at search time.
Very fast retrieval.	Slower but more accurate.
Finds similar documents.	Reorders retrieved documents.
Used before retrieval.	Used after retrieval.

Real Production Flow



Metadata Filtering

What is Metadata Filtering?

Metadata Filtering is a retrieval technique where documents are filtered based on their metadata (attributes) **before or during** retrieval. Instead of searching all documents, we first select only the documents that satisfy specific conditions.

Definition

Metadata Filtering restricts the search space by **applying conditions on document metadata** such as author, date, category, language, source, department, or location before performing retrieval.

What is Metadata?

Metadata is **data about the document**, not the document content.

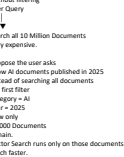
Example document:
Document:
Pinecone is a vector database...

Metadata:
{
 "id": "101",
 "title": "Introduction to Pinecone",
 "author": "John",
 "category": "AI",
 "language": "English",
 "year": 2023,
 "department": "Engineering"
}

Notice:
The metadata is **not** embedded.
Only the document text is embedded.

Why Do We Need Metadata Filtering?

Imagine your vector database contains **10 million documents**.



Architecture



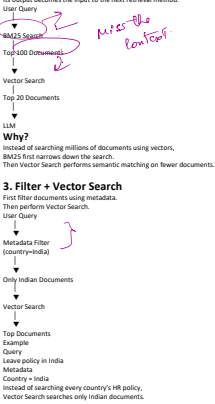
Example

Suppose we have

DOCUMENT 9

2. Sequential Hybrid Retrieval

One retrieval method is executed first. Its output becomes the input to the next retrieval method.



3. Filter + Vector Search

First filter documents using metadata. Then perform Vector Search.

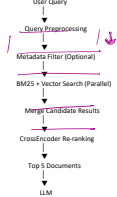


Retrieval Strategy Comparison

Strategy	How It Works	Best Use Case
Parallel Hybrid	BM25 & Vector together	General RAG systems
Sequential	BM25 -> Vector	Large document collections
Filter + Vector	Metadata -> Vector	Enterprise search
Multi-stage	BM25 -> Vector -> Re-ranker	Production RAG
Query Routing	Choose retriever based on query	Multi-domain RAG

Which Strategy is Used in Production?

Most enterprise systems use a **multi-stage hybrid pipeline**.



RAG over APIs

What is API RAG?

API RAG is a type of Retrieval-Augmented Generation where the LLM retrieves information from **APIs** instead of documents.

Document RAG



Example:
What is the leave policy?
Source: HR Policy PDF

API RAG



Example:
What is my leave balance?
GET /leave/101
{
 "leave_balance": 18
}
LLM: You have **18 leave days** remaining.

Why No Chunking or Embeddings?

Because the API already returns the **exact** data.
No need to:
• Chunk documents
• Create embeddings
• Store vectors
• Perform similarity search

What Does the LLM Add?

The LLM:
• Understands the user's intent.
• Chooses the correct API.
• Converts JSON into a natural-language response.
• Can combine data from multiple APIs.

Example

User: Can I take leave tomorrow?
LLM calls:
• Leave API
• Holiday API
Combines both responses and answers:
"Yes, you have 18 leave days remaining, and tomorrow is a working day."

API RAG vs Document RAG

Document RAG	API RAG
PDFs, Docs	REST/GraphQL APIs
Chunking	No Chunking
Embeddings	No Embeddings
Vector DB	No Vector DB
Static Data	Live Data

Key Takeaway

RAG is **not** equal to Vector Database.
RAG = Retrieve external information + LLM generates the answer.

The retrieval source can be:
• Documents (PDFs, Word)
• APIs
• SQL Databases
• Graph Databases
• Elasticsearch

A **vector database** is just **one way to perform retrieval**, not the definition of RAG.

ID	Document	Category	Year
1	Pynecone Introduction	AI	2025
2	Python Basics	Programming	2024
3	LangChain Tutorial	AI	2025
4	React Guide	Web	2025

User adds
LangChain
Without metadata filter
Vector Search checks
1
2
3
4

Context Window Optimization
What is Context Window Optimization?

Definition
Context Window Optimization is the process of selecting, compressing, and organizing the most relevant information so that it fits within an LLM's context window while maximizing answer quality.
Simply put:
Instead of sending **all** retrieved documents to the LLM, we send **only** the most useful information.

Why Do We Need It?
Every LLM has a **context window** (maximum number of tokens it can process).
Example:

Model	Context Window
GPT-4.1	Large context window
Gemini 2.5	Large context window
Claude	Large context window

Even with large context windows, sending unnecessary information:

- Increases cost
- Increases latency
- Can confuse the model
- Reduces answer quality

Example
Suppose your Vector DB returns
Document 1 → 1000 tokens
Document 2 → 1500 tokens
Document 3 → 1200 tokens
Document 4 → 800 tokens
Document 5 → 900 tokens

Total
5400 tokens
But only
1200 tokens
are actually useful.
Instead of sending
5400 tokens
we optimize and send
1200 tokens

Without Optimization



With Context Window Optimization



Common Context Window Optimization Techniques

1. Top-K Retrieval

Instead of sending
20 documents
Send
Top 5 documents
Example
results = index.query(
 vector=query_embedding,
 top_k=5
)

2. Re-ranking

Hybrid Search returns
20 documents
Re-ranker selects
Top 5
Only those go to the LLM.

3. Chunk Selection

Suppose a document has
100 paragraphs
The answer exists only in
Paragraph 62
Instead of sending
100 paragraphs
Send only
Paragraph 62

4. Context Compression

Original
The annual leave policy was introduced in 2022...
(long explanation)
Employees receive 20 days of leave.
(More history...)
Compressed
Employees receive 20 days of annual leave.
The important information remains.

5. Remove Duplicate Chunks

Retrieved
Chunk 1
Pinecone is a vector database.
Chunk 2
Pinecone is a vector database.
Chunk 3
Pinecone stores embeddings.
Instead of sending
1
2
3
Remove duplicates
Send
1
3

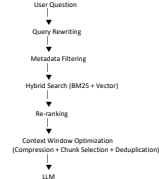
6. Metadata Filtering

Instead of searching
10 Million Docs
Search only
HR Documents
Less context.
Better retrieval.

7. Query Rewriting

Original
How does it work?
Rewrite
How does Pinecone Hybrid Search work?
Better retrieval
↳
Less unnecessary context.

Production RAG Pipeline



Simple Python Example

Suppose we retrieved
retrieved_docs = [
 "Pinecone is a vector database.",
 "Pinecone is a vector database.",
 "BM25 performs keyword search.",
 "Hybrid Search combines BM25 and vectors."
]


```
Remove duplicates
optimized_context = {subject.fromkeys(retrieved_docs)}
print(optimized_context)
Output
{
  "Pinecone is a vector database.",
  "BM25 performs keyword search.",
  "Hybrid Search combines BM25 and vectors."
}
Now send
context = "q" + join(optimized_context)
response = fin.invoke(
  r"""
Context:
{context}
Question:
What is Hybrid Search?
"""
)
Instead of sending repeated information, the LLM receives a cleaner context.
```

Real Enterprise Optimization

Imagine the retriever finds
50 Documents
↓
Re-ranker
Top 10
↓
Chunk Selector
Top 6 Chunks
↓
Context Compressor
3 Short Chunks
↓
LLM
This pipeline reduces tokens while preserving the information needed to answer the question.

Embedding Optimization
What is Embedding Optimization?
Definition
Embedding Optimization is the process of improving how text is converted into vector embeddings so that semantically similar documents are retrieved more accurately and efficiently.
In simple words:
Instead of simply calling
embedding = model.encode(text)
we optimize what we embed, how we embed it, and which model we use to improve retrieval quality.

Why Do We Need Embedding Optimization?
Suppose you have two documents:
Doc1:
Pinecone is a vector database.
Doc2:
Apple released iPhone 16.
User asks
What is a vector DB?
If your embeddings are poor, the retriever might return
Apple released iPhone 16.
Instead of
Pinecone is a vector database.
Embedding optimization improves retrieval accuracy.



Common Embedding Optimization Techniques

1. Choosing the Right Embedding Model
★★★★★

The biggest optimization:
Poor Model
SentenceTransformer("all-MiniLM-L6-v2")
Better Model
text-embedding-3-small
Even Better
text-embedding-3-large
Different models produce different quality embeddings.

Example
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
embedding = model.encode(
 "What is Hybrid Search?"
)
Changing the model changes retrieval quality.

2. Clean the Text Before Embedding

Bad
***** Pinecone ##### is a vector database!!!!
Clean
Pinecone is a vector database.
Cleaner text usually leads to better embeddings.
Example
import re
text = "***** Pinecone ##### is a vector database!!!!"
text = re.sub("[^A-Za-z0-9]+", "", text)
print(text)
Output
Pinecone is a vector database

3. Chunk Optimization ★★★★★

Instead of embedding a huge document
5000 words
Split it into chunks.
Example
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
 chunk_size=500,
 chunk_overlap=100
)
chunks = splitter.split_text(document)
Then
embeddings = model.encode(chunks)
Better retrieval.

4. Add Metadata Before Embedding

Instead of embedding only
Leave Policy
Embed
Department: HR
Leave Policy
Now the embedding contains more context.
Example
text = "I"
Department: HR
Leave Policy
Employees receive 20 days leave.
"""
Better semantic representation.

5. Query Optimization

Document
Hybrid Search combines BM25 and Vector Search.
User
Hybrid
Rewrite
What is Hybrid Search?
Generate embedding from rewritten query.
Better retrieval.

6. Normalize Embeddings

Many vector databases expect normalized vectors.
embedding = embedding / np.linalg.norm(embedding)
Now every vector has unit length.
This improves cosine similarity calculations.

7. Domain-Specific Embeddings

Medical documents
↓
Medical embedding model
Legal documents
↓
Legal embedding model
Code
↓
Code embedding model
Using a domain-specific model often improves retrieval.

8. Remove Noise

Don't embed

Footer
Page Number
Copyright
Contact

Only embed
Actual Content
Noise reduces embedding quality.

9. Batch Embedding

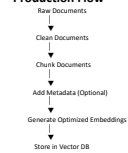
Instead of
for doc in docs:
 model.encode(doc)
Do
embeddings = model.encode(
 docs,
 batch_size=32
)
Much faster.

10. Cache Embeddings

Don't generate embeddings every time.
Store them
embedding_cache[text] = embedding

Huge performance improvement.

Production Flow



Retrieval Performance Tuning

What is Retrieval Performance Tuning?

Definition

Retrieval Performance Tuning is the process of improving the accuracy, speed, relevance, and efficiency of the document retrieval stage in a RAG system.

The goal is to retrieve:

- ☒ More relevant documents
- ☒ Faster results
- ☒ Fewer irrelevant documents
- ☒ Lower latency and cost

Why is it Important?

Imagine you have 10 million documents.

Use asks:

How does Hybrid Search work?

If the retriever returns:

1. Apple iPhone Launch
2. Leave Policy
3. Hybrid Search combines BM25 and Vector Search

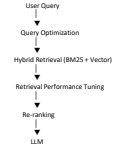
The LLM may generate a poor answer because only one retrieved document is relevant.

Retrieval performance tuning helps retrieve:

1. Hybrid Search combines BM25 and Vector Search
2. BM25 Keyword Search
3. Vector Search

Now the LLM has much better context.

Retrieval Pipeline



Common Retrieval Performance Tuning Techniques

1. Tune Top-K ★★★★★★

One of the easiest optimizations.

Bad

```
results = index.query(
    vector=query_embedding,
    top_k=100
)
```

Too many documents.

Better

```
results = index.query(
    vector=query_embedding,
    top_k=5
)
```

Now only the best documents are returned.

How to choose?

Top-K Effect

- | | |
|-----|----------------------------|
| 1 | May miss important context |
| 3 | Good for FAQs |
| 5 | Common choice |
| 10 | Large enterprise systems |
| 100 | Too much noise |

2. Improve Chunk Size ★★★★★★

Bad

5000-word chunk
Too much information.

Too small

20 words
Not enough context.

Good
300-500 words

or

500-1000 characters

depending on the embedding model and document type.

Example

```
RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)
```

3. Chunk Overlap

Without overlap

Chunk1
Machine Learning is...
Chunk2
...used in AI
Information gets split.

With overlap

Chunk1
Machine Learning is...
Chunk2
Learning is used in AI
Better retrieval.

4. Hybrid Search ★★★★★★

Instead of

Only BM25

Use

BM25 + Vector Search
Better recall.

5. Metadata Filtering

Instead of searching

10 Million Documents

Search only

10 Documents

Faster retrieval.

Example

```
filter=[
    "department": [
        "Sales", "Marketing"
    ]
]
```

6. Query Rewriting

Original

How does it work?

Rewriter

How does Pinecone Hybrid Search work?
Much better retrieval.

7. Re-ranking ★★★★★★

Retriever returns

20 documents

Re-ranker selects

Top 5

Higher quality context.

8. Better Embedding Model

Instead of

Small embedding model

Use

Large embedding model
Semantic retrieval improves.

9. Normalize Embeddings

embedding = embedding / np.linalg.norm(embedding)

Improves cosine similarity consistency.

10. Remove Duplicate Chunks

Retriever

Chunk1

Chunk2

Chunk2

Chunk3

Remove duplicates

Chunk1

Chunk2

Chunk3

More useful context.

11. Multi-query Retrieval

Instead of searching

What is Hybrid Search?

Generate

Explain Hybrid Search

BM25 + Vector Search

Semantic Search

Retrieve using all queries.

Merge results.

This improves recall.

12. Parent-Child Retrieval

Store

Small Chunks

Retrieve
Whole Document
The retriever finds precise matches while the LLM receives richer context.

13. Index Optimization

Choose the appropriate vector index.
Examples

- HNSW
- IVF
- Flat Index

Each offers different trade-offs between speed, memory, and accuracy.

Production Retrieval Pipeline



Index Optimization

What is Index Optimization?

Definition
Index Optimization is the process of choosing and configuring the most suitable indexing algorithm and parameters to improve the speed, memory usage, and retrieval accuracy of a vector database.
In simple words,
Instead of checking every vector in the database, the index organizes vectors so that similar vectors can be found much faster.

Why Do We Need an Index?

Suppose you have 1 million document embeddings.
User asks
What is Hybrid Search?
Without an index
Query Vector

↓

Compare with Doc1
Compare with Doc2
Compare with Doc3
...
Compare with Doc1000000
This is called Brute Force Search.
Time Complexity
O(N)
If there are
1 Million vectors
you compare against all one million vectors.
Very slow.

With Index

Instead
Query Vector

↓

Vector Index

↓

Nearest Neighbors
Now only a few thousand vectors are examined.
Much faster.

Real-Life Analogy

Imagine a library with 10 lakh books.
Without index
Find Python Book

↓

Check every shelf
May take hours.

With index
Computer

↓

Shelf 12

↓

Row 3

↓

Book Found
Much faster.

Architecture



Types of Vector Indexes

1. Flat Index (Brute Force)

Query

↓

Compare with every vector
Example
Q

↓

V1
V2
V3
V4
V5
...

V1000000

Advantages
☒ 100% accurate

Disadvantages
☒ Very slow

Python Example

```
from sklearn.metrics.pairwise import cosine_similarity  
cosine = cosine_similarity  
[query_embedding],  
document_embeddings  
)  
This is Flat Search.
```

2. HNSW (Hierarchical Navigable Small World)

Most popular vector index.
Used by

- Pinecone
- Qdrant
- Weaviate
- Milvus

Idea
Instead of checking every vector
Query

↓

Closest Cluster

↓

Closest Node

↓

Nearest Documents
Like Google Maps:
You don't visit every road.
You follow the best roads.

Advantages
☒ Very fast
☒ Very accurate

Disadvantages
Uses more memory

HNSW Example



The algorithm "navigates" the graph to quickly reach nearby vectors.

3. IVF (Inverted File Index)

Idea
First cluster vectors.
All Documents

↓

Cluster 1
Cluster 2
Cluster 3
Cluster 4
When user searches
Query
↓
Nearest Cluster
↓
Search only that cluster
Instead of storing
1 Million vectors
search
20,000 vectors

Advantages
Fast
Less memory

Disadvantages
May miss some nearest neighbors if the correct vector lies in another cluster.

4. PQ (Product Quantization)

Idea
Compress vectors.
Instead of storing
1536 float numbers
Store
Compressed representation
Memory becomes much smaller.
Useful for
100 Million+
Vectors

Performance Comparison

Index	Accuracy	Speed	Memory
Flat	★★★★★	★	★★★★★
HNSW	★★★★★	★★★★★	★★★★★
IVF	★★★★	★★★★★	★★★★★
PQ	★★	★★★★★	★★★★★

Which Index Does Pinecone Use?

Pinecone automatically chooses and manages the indexing strategy for you. You typically do not configure HNSW or IVF directly in Pinecone.
Other databases expose more index configuration:

Database	Common Index Types
Qdrant	HNSW
Weaviate	HNSW
Milvus	HNSW, IVF_FLAT, IVF_PQ, FLAT
FAISS	Flat, IVF, HNSW, PQ

FAISS Example

```
import faiss
import numpy as np
dimension = 384
# Flat Index
index = faiss.IndexFlatL2(dimension)
# Add vectors
index.add(np.array(docs_embeddings).astype("float32"))
# Search
distance, i = index.search(
    np.array([query_embedding]).astype("float32"),
    k=5
)
print(i)
```

IVF Example

```
quantizer = faiss.IndexFlatL2(384)
index = faiss.IndexIVFFlat(
    quantizer,
    384,
    100
)
index.train(train_vectors)
index.add(docs_vectors)
```

HNSW Example

```
index = faiss.IndexHNSWFlat(
    384,
    32
)
index.add(docs_vectors)
```

Production Pipeline



